



A knowledge-based system for improving the consistency between object models and use case narratives

Narasimha Bolloju^{a,*}, Christoph Schneider^a, Vijayan Sugumaran^{b,c}

^a Department of Information Systems, City University of Hong Kong, Hong Kong

^b School of Business Administration, Oakland University, Rochester, MI, USA

^c Department of Service Systems Management and Engineering, Sogang Business School, Sogang University, Seoul 121-742, Republic of Korea

ARTICLE INFO

Keywords:

Object modeling
Use case narratives
Class diagrams
Model consistency
Knowledge-based systems

ABSTRACT

In today's systems development environments, object models are playing an increasingly important role in contributing to the agility and flexibility expected of the information systems being built. While current computer-aided software engineering tools can aid in creating object models, they do not provide much support in ensuring that the object models created are consistent with the specifications in use case narratives. This paper presents a methodology and a knowledge-based system to facilitate the verification of consistency of a given object model against a set of use case narratives. The methodology is implemented as a prototype knowledge-based extension to an open source CASE tool. The prototype's ability to reliably extract relevant information from use case narratives and its role in verifying the consistency of object models have been evaluated using a laboratory experiment. By analyzing use case narratives utilizing natural language processing techniques and applying collaboration patterns and heuristics, this methodology can determine missing and invalid model elements to guide the analyst in creating object models that are consistent with the requirements specified in a set of use case narratives. The results from this design science research indicate that the prototype system can be a useful tool to assist in this process.

© 2012 Elsevier Ltd. All rights reserved.

1. Introduction

Today's tightly connected world has seen rapid advances in the information systems used by individuals, groups, and business organizations. This progress has fueled rapid changes in information systems development (ISD) methodologies, leading to new and continuing challenges for information systems professionals. ISD makes wide use of use case models to capture information system requirements in terms of use case diagrams and descriptions or narratives, which are then used to develop object models by identifying classes with particular attributes and operations, and by the relationships among those classes. With the increasing popularity of the Unified Modeling Language (UML) and object oriented programming languages, object modeling has become an important step in the systems development process. Object models are extremely useful in supporting activities such as client verification, clarifying technical understanding, programmer specifications and maintenance documentation. Consequently, object

modeling skills are a crucial component of the skill set of today's information systems professionals.

However, acquiring object modeling skills is often challenging, especially for beginners, as the tasks and solutions are ill-defined (Borstler & Hadar, 2008; Moritz & Blank, 2005; Siau & Cao, 2001; Siau, Erickson, & Lee, 2005). Furthermore, a key challenge in developing object models is the complexity of UML, which has been demonstrated to be between two and 11 times more complex than other modeling methods (Siau & Cao, 2001), so that even experienced analysts prefer not to use the UML method due to lack of understanding (Dobing & Parsons, 2006). Computer-aided software engineering (CASE) tools have been developed to support the “mechanics” of creating object models, by aiding in drawing, documenting, and navigating through object models (Eriksson, Penker, Lyons, & Fado, 2004); however, these tools typically lack features that support the development of object models that are consistent with the requirements captured during the early stages of the development process.

This study addresses that shortcoming by focusing on an important research question: *how can CASE tools be enhanced to support validating that the object models created are consistent with the requirements specified in the use case narratives?* The answer to this question lies in the fact that CASE tools need to be augmented with features that are capable of enhancing the development of object

* Corresponding author. Address: Tat Chee Avenue, Kowloon Tong, Hong Kong. Tel.: +852 34427545; fax: +852 34420370.

E-mail addresses: narsi.bolloju@cityu.edu.hk (N. Bolloju), christoph.schneider@cityu.edu.hk (C. Schneider), sugumara@oakland.edu (V. Sugumaran).

models that faithfully represent a set of given system requirements. Thus, the specific objectives of this research are to: (a) develop a methodology for validating object models by using the relevant information from use case narratives specific to the system requirements, (b) implement the methodology in a prototype system, and (c) evaluate the prototype system to understand the extent to which consistency can be verified. Following a design science approach (Hevner, March, Park, & Ram, 2004), we propose and build a knowledge-based system that is capable of analyzing the consistency of a given object model against a set of given use case narratives and providing recommendations for improving consistency of the model against those narratives. Although there are several types of inter-model consistency (Mens, Van der Straeten, & Simmonds, 2005), this study addresses the inter-model consistency of object models evolved from use case narratives during the analysis phase of systems development.

The remainder of this paper is organized as follows. Section 2 provides background information on the challenges faced in object modeling and the available mechanisms for consistency validation. This section also summarizes the capabilities and shortcomings of commonly used UML modeling environments and model checking tools. Section 3 outlines the proposed methodology for assistance in developing object models consistent with a set of use case narratives. Section 4 describes the design and implementation of the prototype system. In Section 5, we report a study conducted to evaluate the prototype system to assess its efficacy in improving the consistency of the object models created. Section 6 discusses the results of the evaluation process. Finally, Section 7 concludes the paper.

2. Background and motivation

Although UML models are widely used in systems analysis and design, creating high quality models has always been a challenge. While different techniques exist within UML for modeling various aspects of systems, many researchers have documented the difficulties and complexities associated with using UML. In this section, we provide a brief review of object modeling, the support provided by UML modeling environments for consistency management, and then discuss support mechanisms that are currently available.

2.1. Object modeling

Object models, comprising classes, the attributes and operations of classes, and the relationships among the classes in the problem domain, have had a relatively long history of use in the fields of object-oriented programming and systems development (Moritz & Blank, 2005). Thus, object models are now widely considered to be the most useful tools for clarifying and understanding the static structure of a system (Eriksson et al., 2004). Object models matching the requirements specifications expressed in use case narratives can minimize communication problems and misunderstandings of requirements, thereby reducing the effort required to accommodate changes at later stages of system development; as a result, object models are an integral part of UML modeling. The object modeling process includes identifying classes, the attributes and operations of classes, and the relationships among the classes in the problem domain, based on requirements that are typically captured in use cases and associated diagrams. The resulting object models are graphically depicted as class diagrams. However, various difficulties encountered in identifying the required model elements (classes, attributes, operations, and relationships) often affect how closely the resulting models reflect the requirements of the system under consideration.

As UML modeling (and, in particular, object modeling) is a cognitively intensive activity, several commercial and open source

UML modeling tools have been developed to support analysts in the “mechanical” aspects of creating UML models (a partial list of the existing tools and their features can be obtained from Wikipedia, 2011). Most of these tools provide a number of basic functionalities that are common in systems modeling, such as diagram editors that assist in building models of systems. However, the current tools are vastly different in regards to ease of use and the level of support they provide in assisting analysts in the development of high quality models. In supporting different tasks, the current tools make use of a variety of features, including visual and declarative editors, support for business process modeling and metadata management, context sensitive entry, syntax coloring, and customizable design elements. Although these tools focus on the entire lifecycle of systems development, they are extremely complex and demand a steep learning curve. As a result, they do not provide adequate support for analysts to quickly develop high quality object models.

Typically, the modeling tasks are iterative processes, and the validation of the models as they are created is an important step in improving the quality of the final models. Because UML models represent system requirements from different perspectives, maintaining consistency across different artifacts is a major concern. Moreover, maintaining consistency between multiple UML diagrams that represent the static and dynamic aspects of a system is a non-trivial task, particularly when the models are frequently changed (Kotb & Katayama, 2005). Although existing tools perform reasonably well in checking individual models, such as in verifying UML activity diagrams (Eshuis & Wieringa, 2004), they have been shown to be cumbersome in checking for consistency and providing quality and timely feedback (Egyed, 2006). Hence, Lange (2006) advocates the use of metrics and visualization techniques to improve the quality of UML models during development. Similarly, Egyed (2004) discusses a tool called a UML/Analyzer that automatically determines the most appropriate consistency checking rules to execute during model changes to detect inconsistencies and provide instant feedback to the designer. Yet, those tools primarily focus on syntactic checking. Two notable exceptions are the tools described by Campbell, Cheng, McUmbert, and Stirewalt (2002) and Nentwich, Capra, Emmerich, and Finkelstein (2002). The tool described by Campbell et al. is capable of performing structural and behavioral analyses to detect errors, which it presents in terms of the original UML diagrams. In addition, it performs intra- and inter-diagram consistency checks as part of the analysis process. The tool described by Nentwich et al. provides an incremental way of finding inconsistencies in UML diagrams represented as XML files. Despite these efforts, in their comprehensive review of research prototypes for maintaining UML model consistency, Lucas, Molina, and Toval (2009) highlight a lack of research devoted to tackling inter-model consistency problems and integrating consistency management features within CASE tools.

Creating quality object models goes beyond the mechanical aspects of drawing syntactically correct diagrams. In particular, although the quality of a model is heavily dependent on the extent to which it captures a system's requirements, UML modeling tools do not provide explicit mechanisms to help systems analysts validate that the models they create truly capture the requirements specified. Because an object model is a form of conceptual model, the challenges associated with object modeling are comparable to those associated with conceptual modeling. When developing conceptual models, novice or inexperienced systems analysts tend to encounter difficulties in the areas of domain-specific knowledge and problem-structuring, as well as in the cognitive processes associated with identifying model elements (Schenk, Vitalari, & Davis, 1998); especially for novice analysts, conceptual modeling is a complex task to perform efficiently and effectively, because of the absence of standardized validation procedures (Shanks, Tansley, &

Weber, 2003). Insufficient support provided by CASE tools is also evidenced by the observation by Gelbard, Te'eni, and Sade (2010), who found that in less than 20% of the systems development projects required classes were identified during the analysis phase. Furthermore, requirement engineers or systems analysts' lack of understanding of the domain and miscommunication between users and technical persons are two common problems associated with the early stages of systems development (de la Vara & Sánchez, 2008). In sum, the effectiveness of a conceptual model can be affected by the task requirements, the analyst's modeling experience and cognitive abilities, the interpreter's prior exposure to conceptual models, and the complex relationships that exist between modeling constructs (Wand & Weber, 2002). Assistance provided to analysts to address one or more of these challenges is likely to contribute towards creation of better object models.

2.2. Support mechanisms for object modeling

Given these challenges to creating object models, researchers have suggested various approaches for supporting object modeling. The prevalent approaches for supporting object modeling can be broadly classified as (a) guidance focused approaches and (b) critique focused approaches, and c) generation focused approaches.

Guidance focused approaches include mechanisms such as practitioner-oriented recommendations for developing quality models, as well as analysis patterns and frameworks for specific domains and wizards. Practitioner recommendations typically focus on tasks such as identifying classes and relationships, following naming and presentation conventions, and using analysis patterns (e.g., Batra, 2005; Bolloju, 2004) and frameworks. Other guidance focused approaches provide reusable frameworks constructed by domain engineers that can be customized to suit the target environment (e.g., Hakala et al., 2002; Morisio, Travassos, & Stark, 2000; Viljamaa, 2001). Whereas in the simplest form, guidance focused approaches are provided in the form of written documentation, they can also be built into automated tools. For example, Sugumaran and Storey (2006) use domain ontologies to guide novice and experienced modelers in creating complete and consistent database designs. Relatedly, using a wizard-based approach, Antony and Batra (2002) developed a rule-based expert system called CODASYS that addresses the cognitive problems novice analysts often face by using question dialogs and warning messages to reduce errors, particularly in deriving correct relationships.

Critique focused approaches emphasize providing advice either during or after the development of a conceptual model, primarily through the modeling tools. For example, ArgoUML (Robbins & Redmiles, 1998, 2000) incorporates a set of critiquing agents that lower the cognitive load of software designers by continuously checking an existing model for potential problems and providing advice on areas of improvement. Sugumaran and Storey (2002) report an application that uses domain ontologies to validate a given data model and suggest missing entities and relationships.

Finally, *generation focused approaches* include a variety of mechanisms for directly creating or translating object models from textual descriptions of system requirements or through question-and-answer sessions. For instance, Wohed (2000) developed a prototype of an expert system conceptual modeling "wizard" that gathers user responses to a sequence of six questions to incrementally build a conceptual model for the reservation domain. Other approaches use natural language processing to translate requirements into conceptual models (Kaindl, 2004; Overmyer, Lavoie, & Rambow, 2001; Purao, 1998). Examples of these tools include AP-SARA, a web-based prototype that creates object-oriented designs from simple requirement descriptions written in natural language using heuristics and a patterns database (Purao, 1998) and RETH, a tool for requirement engineering that uses natural language pro-

cessing to generate associations and specialization relations between objects based on natural language definitions of classes (Kaindl, 2004). Similarly, Overmyer et al.'s tool employs natural language processing to extract words from a textual document, to identify the part-of-speech and base forms analysts' require to tag these words as classes, operations, or attributes, and to then generate a corresponding object model. In a related example, Purao, Storey, and Han (2003) developed and empirically evaluated an intelligent assistant that generates initial conceptual designs for later refinement and incorporates learning mechanisms for enhancing analysis pattern reuse. Bolloju and Sugumaran (2012) implemented a combination of guidance-focused and critique-focused approaches in a prototype system that provided recommendations mostly based on the patterns manually assigned to various relationships.

Each of the three types of approach outlined above has certain strengths and limitations. The guidance focused approaches provide exhaustive lists of *dos* and *don'ts*, yet few of these have been incorporated into CASE tools; thus, systems analysts require support in dealing with the large number of guidelines to benefit from this type of approach. Critique focused approaches are normally expected to identify errors in a given model and provide recommendations, but mostly provide broad sets of generic recommendations in the form of lengthy checklists, typically drawn from guidance based approaches. Finally, generation focused approaches appear to be attractive due to automation, but heavily depend on the quality of inputs (such as textual descriptions of the requirements used) and do not scale up well to real life systems.

Overall, although the various commercial and open source tools available to aid analysts with the mechanics of drawing UML models offer some forms of model validation, and a number of research prototype consistency management tools have been built, they have limited capacity for helping analysts create quality object models based on a given set of system requirements. In this research, we focus on the consistency aspect of object model quality, which requires the model to be complete and valid with respect to the requirements specified in a set of use case narratives. Specifically, we believe that the cognitively intensive process of object modeling can be facilitated by integrating critique based support into CASE tool environments to provide problem specific recommendations for improving the consistency between a given object model and a set of use case narratives.

3. Proposed methodology

A key goal of this study is to provide assistance during the development of object models or class diagrams that are consistent with the textual requirements specified in a set of use case narratives. Specifically, our aim is to help systems analysts overcome the cognitive challenges associated with validating the consistency of object models against the system requirements captured in use case narratives. Hence, a key outcome of this study is the creation of a prototype system that provides an initial step towards achieving this goal. Our research largely follows a design science approach. As opposed to natural sciences, which attempt to gain a deeper understanding of a phenomenon, design science focuses on developing tangible artifacts to serve designated purposes, the outcomes of which are evaluated according to their utility (March & Smith, 1995). To ensure the relevance and rigor of our research, we considered the business needs and theoretical foundations associated with the development and evaluation of the artifact (Hevner et al., 2004). In addition, we followed a multi-step approach (see Kuechler & Vaishnavi, 2008; Peffers, Tuunanen, Rothenberger, & Chatterjee, 2007) consisting of problem identification, definition of design objectives, the design process,

and evaluation. In this section and the following section, we delve deeper into the proposed methodology for validation of object models against the requirements specified in use case narratives and present the details of the implementation of the prototype system developed to support the methodology.

An information systems development project typically involves many stakeholders who often provide fuzzy and conflicting requirements. As a result, the automated generation of object models remains elusive, and the systems analysts play an important and critical role in translating the requirements into object models. Therefore, rather than attempting to automatically generate object models by making use of a given set of use cases, our proposed

methodology focuses on supporting systems analysts during the object model creation process.

Specifically, our methodology can be classified as a combination of critique and guidance focused approaches, in which both use cases and object models are analyzed to identify missing and invalid elements (i.e., classes, attributes and relationships) in the current version of the object model and to provide specific recommendations or guidance for improvement; the analyst can then review and implement the recommendations based on his or her judgment (see Fig. 1). Following the creation of use cases and a draft object by the systems analyst (i.e., the activities “create use case diagram,” “develop use cases,” and “create draft object model”), our

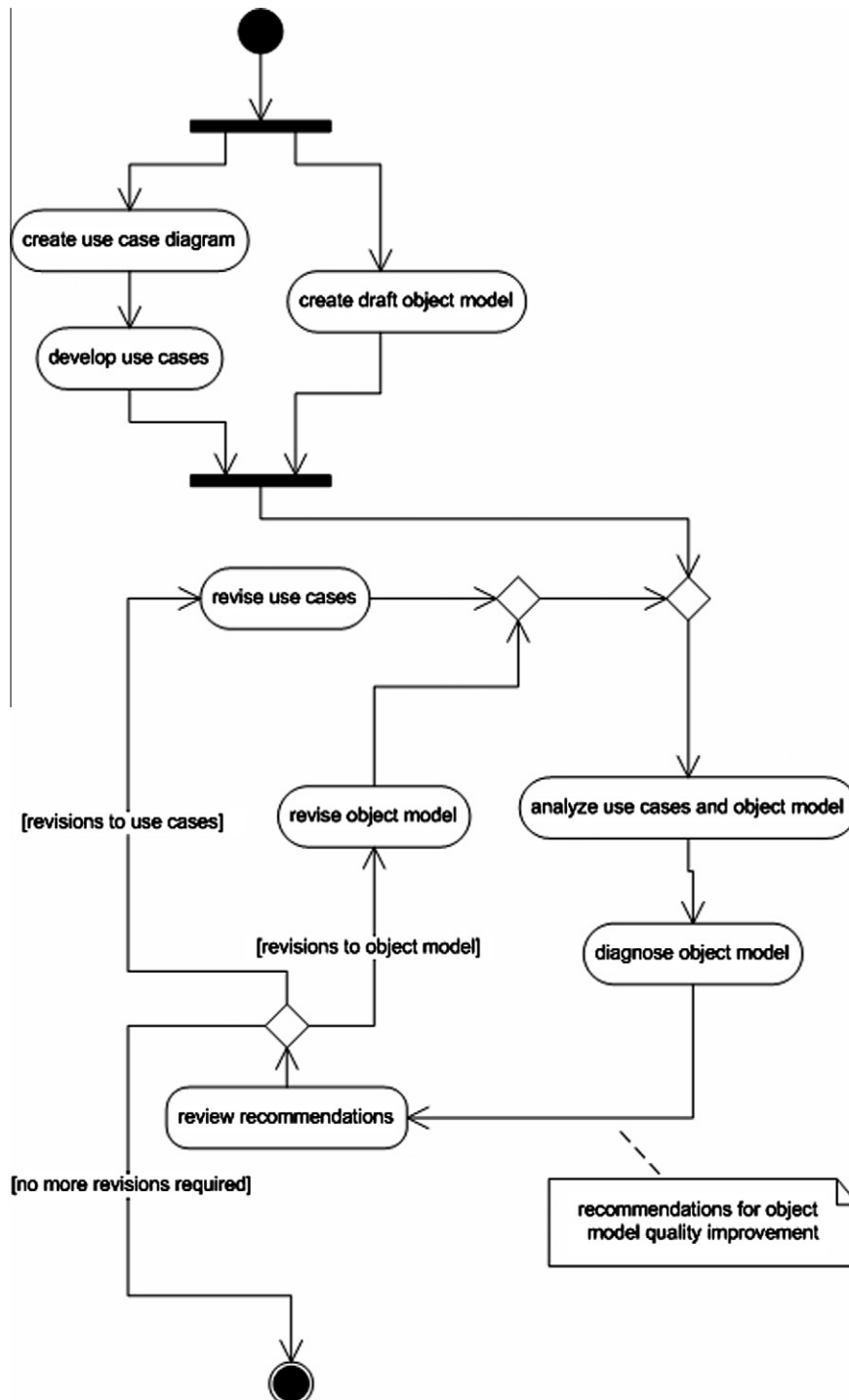


Fig. 1. Activity diagram for the proposed methodology.

methodology starts with analyzing textual material in use cases to identify or recognize key elements and identifying different types of classes in the current version of the object model (the activity “analyze use cases and object model”); this activity completes the preparatory work required for the diagnostic analysis of the current version of the object model based on the given set of use cases.

Using inputs from the previous activity, the “diagnose object model” activity primarily involves the identification of missing and invalid elements in the object model using collaboration pattern knowledge and the provision of related recommendations for adding missing elements and/or removing invalid elements. The recommendations provided need to be reviewed by the analyst to identify and incorporate those deemed most useful for ensuring that the object model created is consistent with the use case narratives. While the useful recommendations mostly require changes to the current version of the object model, certain recommendations may require revisions to the use cases, especially when elements that are valid in the problem domain are considered invalid. The five activities in the lower portion of Fig. 1 are to be repeated as long as there are recommendations that are considered relevant and useful for the given problem domain. The proposed methodology, thus, supports the iterative and incremental tasks that are normally performed during the object modeling process.

The “diagnose object model” activity is based on several heuristics that have been primarily derived from practitioner guidelines, as well as from data modeling and object modeling processes employed by experts. We have performed an exhaustive search of data modeling and object modeling guidelines from popular textbooks and practitioner websites, observed the object modeling process used by experienced analysts (especially based on a given set of use case narratives), and employed commonly occurring analysis patterns in identifying a set of heuristics for implementation. The following examples illustrate two of the heuristics used in the diagnostic process to identify missing relationships and missing transaction classes.

(1) Identifying a missing relationship:

There is a missing relationship between a pair of distinct classes *c1* and *c2*

IF there does not exist a relationship that connects *c1* and *c2*

AND

there exist two pattern players *p1* and *p2* derived for *c1* and *c2* respectively

such that $\langle p1, p2 \rangle$ is a valid collaboration pattern

AND

there exist two recognized entities *e1* and *e2* from the use cases

such that *e1* maps to *c1* and *e2* maps to *c2* and

e1 and *e2* belong to the same use case narrative

(2) Identifying a missing transaction class between a pair of classes:

There is a missing transaction class between a pair of classes *c1* and *c2*

IF *c1* has a pattern player *p1* of type *role* and *c2* has a pattern player *p2* of one of *specific item*, *place*, *item*, *line item* types

AND

there does not exist a class *cT* that directly connects with *c1* and *c2*

such that *cT* is a composite transaction class, and *p2* is a *place*, OR

cT is a transaction class, and *p2* is either a *specific item*, *place*, or *item*

AND

there does not exist a composite transaction class *cU* and a *line item* class *cL*

such that *c1* is connected with *cU* and *cU* is connected with *cL* and *cL* is connected with *c2*

and *p2* is either a *specific item* or *item*

AND

there are recognized entities *e1*, *e2* from use case narratives

such that *e1* maps to *c1*, *e2* maps to *c2*, and

e1 and *e2* belong to the same use case narrative

Both of these heuristics are based on patterns, i.e., named blueprints of solutions to a given problem that can be applied to other contexts (Larman, 2001). In other words, patterns are codified solutions that have been shown to be effective in solving a particular type of problem. Appendix B provides an overview of the collaboration patterns and Appendix C illustrates how the first heuristic works on an example use case narrative. The first heuristic determines whether a relationship is required between any given pair of classes in the object model, considering the pattern type of those classes (e.g., *role* and *transaction*) and a corresponding collaboration pattern (e.g., most *role* classes are associated with *transaction* classes), provided any entities of similar types are present in a single use case. For example, if the object model has the two classes *Agent* and *Sale*, an association is required if some use case has *Agent* as an actor and has certain steps referring to *Sale*.

The second heuristic helps in identifying whether a transaction class (either an associative or a composite type) is missing between a pair of existing classes. The first case refers to a possible missing associative transaction class between a *role* type of class and a *specific item*, *line item*, or *place* type of class, unless there is already a composite transaction class connecting the two classes under consideration. The second case is for identifying a missing composite transaction class between the *role* and a *specific item* or *line item* type of class.

To effectively facilitate the development of consistent object models, it is necessary that major portions of the proposed methodology described above must be supported directly within a CASE tool environment. In the following section, we present details of the architecture and implementation of this methodology into an existing CASE tool as a prototype system.

4. Prototype system architecture and implementation

To support the proposed methodology, we developed a prototype knowledge-based system, called OMA (Object Model Analyzer), as an extension to the open-source CASE tool ArgoUML (2006). This prototype provides mechanisms for specifying use case narratives for the use cases present in a use case diagram and invoking a knowledge-based system component to obtain recommendations for improving the consistency of object models created using ArgoUML functionality. Thus, the entire methodology presented in Fig. 1 is support the prototype either through original ArgoUML functionality or through the extensions implemented.

The OMA was implemented using several open source software tools and libraries and developed using Java 1.6 and Jess (Friedman-Hill, 2005). ArgoUML was selected as the basis for the prototype because it is a popular UML tool that provides a set of application programming interfaces to extend its functionality in the form of extension modules and can be easily integrated with required external Java libraries. In addition, we used various other open source tools, such as Protégé (ontology editor), Jess (inferencing engine), and JessTab (to integrate Protégé and Jess) to implement the prototype.

Fig. 2 depicts the architecture of the OMA, which consists of four primary components: (a) use case narrative support, (b) adapter, (c) knowledge base, and (d) processors. The first component offers an interface for adding narratives for the use cases present in a use case diagram. The adapter component facilitates the interaction between ArgoUML and the components of the OMA and connects ArgoUML with the knowledge base system. It also helps in translating object models in ArgoUML data structures into ontology instances and controlling the interaction between ArgoUML and other system components. The knowledge base component maintains the collaboration pattern knowledge and diagnostic rules, which are expressed as a combination of an ontology and a set of Jess rules. OWL ontology was used to represent use case nar-

atives, the object model under development, and the collaboration patterns.

Typical usage of the OMA requires the user to start a project in ArgoUML with a use case diagram and to create narratives for each use case present in that diagram in terms of trigger, pre- and post-conditions, and descriptions of the steps in the main and alternate scenarios. OMA extension facilitates creation of these narratives by the user using structured templates. The user may then create an initial draft of an object model, either parallel to or after the use case specification, using ArgoUML's built-in functionality. Following this, the user can invoke the "object model analyzer" function, which integrates the "analyze use cases and object model" and "diagnose object model" activities (see Fig. 1). The object model

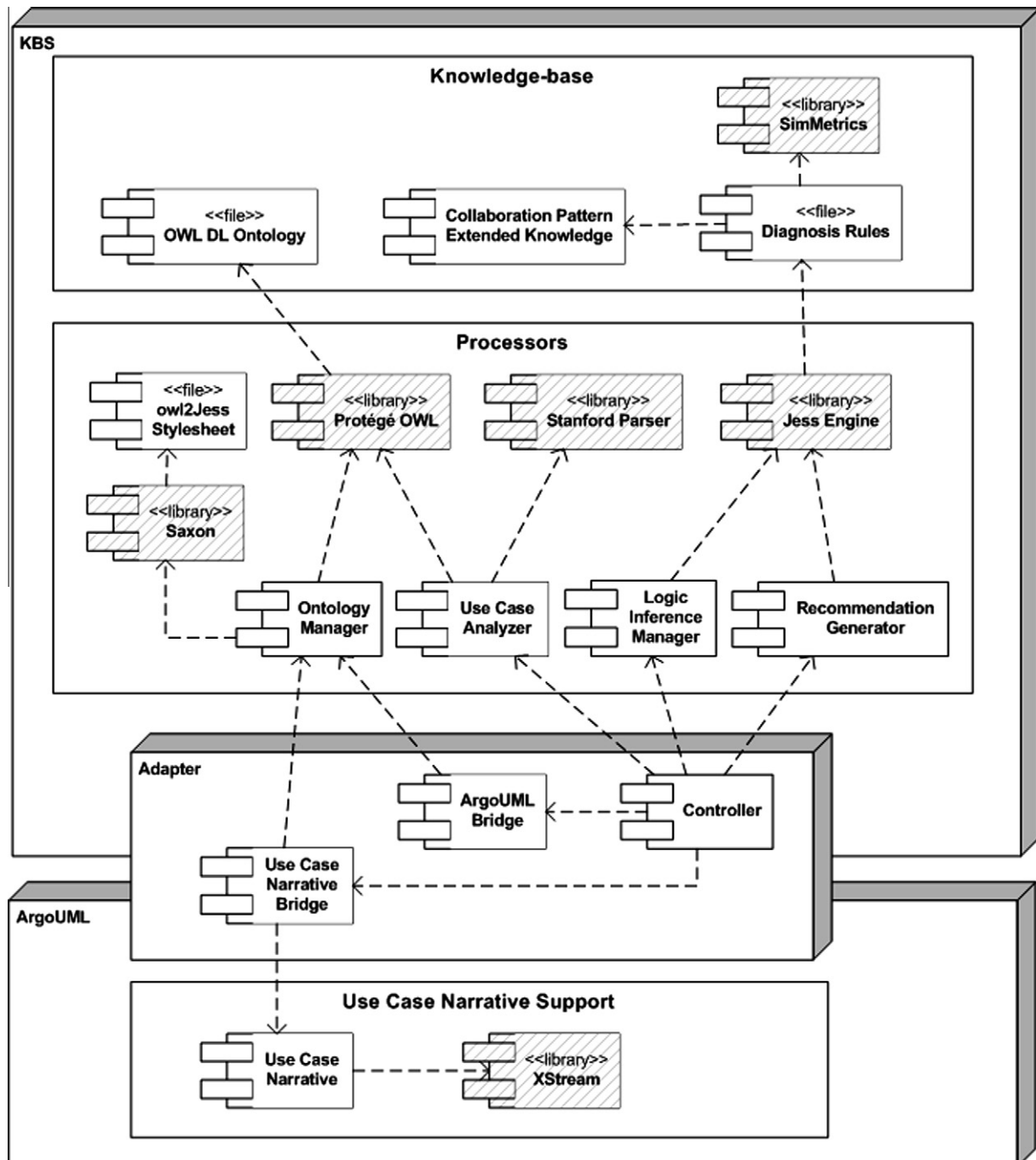


Fig. 2. Architecture of the OMA.

analyzer extracts relevant information from the use cases using natural language processing (NLP) techniques and determines possible pattern player types for each class in the object model. Then, it makes inferences to identify missing and invalid elements in the draft object model, and finally, generates recommendations for making the object model consistent with the narratives. The user is expected to proceed through the activities of reviewing the rec-

ommendations, applying any relevant recommendations to the object model, and invoking the object model analyzer function, as long as applicable recommendations are found. The current version of the prototype does not provide any mechanism to flag inapplicable or incorrect recommendations.

When the user invokes the object model analyzer within Argo-UML, the processor components: (a) convert use cases and the cur-

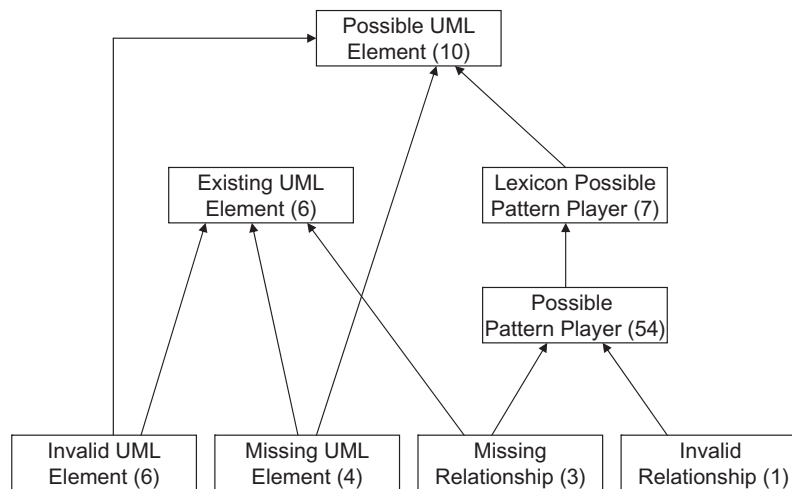


Fig. 3. Groups of heuristics with dependencies.

Analysis Results			
Reports			
Missing Classes and Attributes Missing Relationships Invalid Classes and Attributes Invalid Relationships			
☐ Missing Classes and Attributes			
UML classes and attributes missing from the diagram			
CF ▾	Name	Type	
0.300000	new submission	UMLClass	
0.300000	review decision	UMLClass	
0.100000	id	Attribute	
0.100000	id	UMLClass	
☐ Invalid Classes and Attributes			
UML classes and attributes that are not valid			
CF ▾	Name	Type	
0.256250	Assistant	UMLClass	
0.256250	department	Attribute	
0.256250	designation	Attribute	
0.256250	newAttr	Attribute	
0.256250	noOfCopy	Attribute	
0.256250	noOfMonthIssue	Attribute	
0.256250	skills	Attribute	
0.256250	Staff	UMLClass	
0.256250	StaffID	Attribute	
0.125000	affiliation	Attribute	
0.125000	affiliation	Attribute	
0.125000	bankName	Attribute	
0.125000	cardNumber	Attribute	
0.125000	Checks	UMLClass	
0.125000	dataEntered	Attribute	
☐ Missing Relationships			
Missing either a direct relationship between two existing classes or an indire...			
CF ▾	ClassName	ClassName	Type
0.990528	Assistant	Author	via a Transaction...
0.980330	Assistant	Editor	directly
0.946875	Advisor	Assistant	via a Transaction...
0.913648	Author	Credit Card	via a Transaction...
0.888623	Article	Credit Card	directly
0.880482	Advisor	Credit Card	via a Transaction...
0.859185	Article	Credit Card	via a Transaction...
0.841781	Credit Card	Editor	directly
0.759342	Assistant	Credit Card	directly
0.729598	Credit Card	Subscription	directly
0.583392	Author	Credit Card	directly
0.567769	Advisor	Editor	directly
0.567769	Author	Editor	directly
0.448880	Credit Card	MagazineIssue	directly
0.440000	Credit Card	Subscription	directly
☐ Invalid Relationships			
Relationships that should not exist between classes			
CF ▾	ClassName	ClassName	
0.200000	Payment	Subscription	
0.200000	Receipt	Subscription	
0.200000	Subscription	Payment	
0.200000	Subscription	Receipt	

Fig. 4. An example of recommendations provided.

rent version of the class diagram into OWL ontology, (b) analyze use cases and create an ontological representation of actions and recognized entities, (c) optionally, connect inputs related to archetype assignments (*People, Events, Places, Things*) to recognized elements in the use cases and the class diagram, (d) transform the current ontology into facts in the format required by Jess, (e) perform inference for completeness and validity by applying heuristics to the facts loaded from the Jess file, (f) report suggestions regarding missing and invalid classes, attributes and relationships, and (g) save the ontology as an XML file and the Jess facts as a text file for subsequent use.

As can be seen from this description, the two key activities of the system are analyzing the use cases and performing a diagnostic process that involves applying the heuristics. The Use Case Analyzer uses NLP techniques to analyze a given set of use case narratives and then transform the analyzed information into an intermediate form that works as the basis for the object model analysis and the glossary management. This process uses Stanford Parser library components to parse different parts of the use case narrative to: (1) generate a syntactic parse tree that can be used to find possible glossaries in the use case narratives; (2) find the base form of each token (through word stemming) in the parse tree so that different variants of the same token can be compared; and (3) generate typed dependencies among the tokens. The intermediate form is an OWL ontology, which describes the relationships between subjects, verbs, and objects in the use cases. The Use Case Analyzer processes use cases by analyzing each step in every scenario. For each step, it generates a sub-network of the ontology. After translating the ontology into facts, the system proceeds with diagnosing the object model. The component diagram shown in Fig. 3 depicts the relationships among different heuristics groups, with the number of heuristics in each group implemented in the current version of the prototype. Confidence factors (CF) are associated with several of the heuristics (default CF = 1) to indicate their strength of inference. As part of the inferencing process, the CFs are then combined and propagated for the final recommendation.

Based on the results of the diagnostic process, the system provides feedback (in the form of recommendations) to the user. A sample of recommendations provided for missing and invalid elements is shown in Fig. 4. Each row in the sub-panels contains a recommendation for missing classes and attributes, missing relationships, invalid classes and attributes, and invalid relationships. Each recommendation also includes details, such as whether a direct or an indirect relationship is required, and the confidence in that recommendation, represented by a CF between 0 and 1.

5. Evaluation

Our initial evaluation of the OMA involved testing its effectiveness in analyzing use cases and applying the diagnostic heuristics. We used several sets of use cases and draft object models belonging to different business application domains for this pilot testing. Based on these tests, we refined the use case analyzer component and the diagnostic rules and incorporated significant changes to the user interface components (e.g., presentation of diagnostic results as tables rather than in the form of text) based on the feedback obtained from the testers.

Having established that the system functioned as expected, we proceeded to conduct another, more formal evaluation. Specifically, we conducted a controlled experiment using undergraduate students enrolled in a systems analysis course to evaluate the effectiveness of recommendations provided by the prototype system. Due to the time constraint of two hours available for subjects,

this evaluation focused only on revising object models in one iteration and not on modifying use case narratives in multiple iterations as described in the overall methodology. We believe that these participants can be considered as novice systems analysts who are likely to develop object models with relatively greater inconsistencies compared to experienced analysts, and consequently provide better opportunities for evaluating the strengths and weaknesses of the prototype system.

5.1. Participants

A total of 153 undergraduate students studying information systems in a business college at a large East Asian metropolitan university participated in the experiment. The average age was 21.1 years and 59.5% were male. The students were enrolled in a systems analysis course that mainly covers the object-oriented approach with selected UML techniques. All these participants have completed courses covering information systems fundamentals and have had some programming experience. The class was divided into five laboratory sections, with three sections (92 students) randomly selected to serve as the treatment group and two sections (61 students) as the control group. The study was conducted toward the end of a 13-week course. By this time, the students had become reasonably familiar with object modeling while working on a running case study in the laboratory sessions and another case study in their project work. The study was conducted as an exercise during a 100-min laboratory session and contributed 5% to the students' grade.

5.2. Procedure

The participants in each laboratory section were provided with a case study of a journal subscription system (*SmallBytes*, see Appendix A), along with a use case diagram and a set of six corresponding narratives. Based on the material provided, the participants were instructed to create a draft object model, followed by a revised model, after a short break. Specifically, in addition to a questionnaire on the participants' prior knowledge and skills relating to object modeling, each of the laboratory sessions included the following activities and another 10 minutes for filling the questionnaires.

- (1) The instructor provided a handout of general instructions about the experiment and explained the procedures (10 min).
- (2) Each participant developed an object model (version 1) for the *SmallBytes* subscription system using ArgoUML (35 min).
- (3) Break – Participants were not to discuss their solutions during the break (10 min).
- (4) Students in the treatment groups received object modeling guidelines as well as feedback on the object model from the OMA. Based on this feedback, the participants were given time to revise their initial version of the object model (35 min)
OR
Students in the control groups received only the object modeling guidelines and were given time to revise their initial version of the object model (35 min).

Each participant submitted their initial and final versions of the object models immediately after activities 2 and 4, respectively. After accounting for missing versions and excluding any corrupted or duplicate submissions, we obtained 135 usable pairs (out of 153) of object models. Thus, the usable data set comprised 84 pairs of object models for the three treatment sections and 51 pairs for

the two control sections. We set aside 20 and 10 randomly selected pairs of object models from the treatment and control conditions respectively for training and coding purposes, leaving 105 pairs for the data analysis.

5.3. Analysis of object models

We assessed the consistency of the initial and the revised object models, against the supplied use case narratives, to identify specific improvements made to the object model based on the recommendations from the system. Considering the fact that the object models created (draft and final versions) could have several variations and the large number of object models to be assessed, we employed a two-stage assessment process to match each object model with a model solution. The first stage involved automated assessment using a specially developed program that matched classes, attributes and relationships within a given object model against the model solution. The automated assessment was designed to account for alternative names for elements (e.g., check vs. cheque), as well as for differences in capitalization.

The second stage involved manually resolving any unmatched elements in the class diagrams. This stage supplemented the automated assessment by providing opportunities for validating alternative representations of object models for the given problem domain. Two assessors were trained to verify the automated assessments. Specifically, the assessors confirmed whether the system had matched the elements correctly and, in the case of an incorrect matching, the assessors matched the element with another element, if possible. Any elements not matched by the automated tool were manually matched, if possible.

Each assessor completed this manual assessment process for ten object models from the training set. As this initial assessment showed a high level of inter-assessor agreement (Cohen's kappa 0.865, standard error 0.041), the assessors proceeded to independently perform the manual assessment of the remaining object models. The first assessment stage correctly matched over 70% of the model elements. Both assessors only needed to match the remaining element; this helped to increase the efficiency of the second assessment stage, while minimizing subjectivity.

As some of the initial object models were of very poor quality (e.g., containing very few classes and relationships and, in some cases, no attributes), we dropped pairs corresponding to the initial object models in the bottom 25 percentile with fewer than 4 classes, 3 relationships and 10 attributes, leaving a total of 78 model pairs (treatment group: 41; control group: 37) for statistical analysis. By excluding outliers belonging to both the treatment and the control groups, we believe that we have selected only the object model pairs in which the participants have made a reasonable effort in creating draft object models, thereby resulting in an effective comparison of the final versions of the object models created.

To assess the usefulness of the system's recommendations, two experienced assessors were trained to assess whether the system's recommendations helped in improving the consistency between the use case narratives and the initial object models. The assessors were provided with the expected solution and its possible variations, and categorized each recommendation as 'useful', 'not useful' and 'may be'. They also identified the feedback that had actually been incorporated into the revised version. We tested inter-assessor reliability using a small set of diagrams as part of the training process to ensure that the subjectivity in categorization was minimized (Cohen's Kappa = 0.81).

5.4. Results

A typical object model created included 13 classes, 36 attributes and 13 relationships. The prototype system produced on average

53.56 recommendations regarding missing or invalid elements. Specifically, whereas only a few recommendations were offered for missing classes ($M = 2.66$, $SD = 1.39$), missing attributes ($M = 1.07$, $SD = 1.01$), invalid classes ($M = 6.07$, $SD = 1.82$), and invalid relationships ($M = 1.41$, $SD = 2.03$), the system produced relatively large numbers of recommendations related to missing relationships ($M = 19.76$, $SD = 12.35$) and invalid attributes ($M = 22.59$, $SD = 7.05$). Our analysis showed that, overall, the system was most effective in identifying invalid relationships (95.0%), followed by missing attributes (70.4%) and missing classes (65.9%). The system was considerably less accurate in identifying invalid attributes (38.9%), invalid classes (35.7%), and missing relationships (3.4%). Table 1 summarizes the recommendations and the assessments of their usefulness.

A detailed analysis of the object models created in the first iteration and the specific recommendations offered by the system was performed to understand the specific strengths and weaknesses of the prototype system. While the system offered several useful recommendations for missing *transaction* and *item* types of classes, it was not that effective in identifying *role* types of classes and variations in labeling of *transaction* classes. We found that smaller numbers of missing attribute recommendations were reported partly due to the nature of the reporting mechanism, which listed only the attributes missing in the identified classes. Large numbers of missing relationships were recommended due to prototype limitations with respect to not effectively pruning associations between certain classes (either directly or via a *transaction* class), and providing recommendations with a low confidence factor, primarily involving association between a *transaction* class with *role*, *item* and other *transaction* classes. Furthermore, some recommendations for missing associations between a pair of classes were provided twice – both directly and with a *transaction* class.

As anticipated, the prototype performed better in identifying invalid classes, attributes and relationships, as the process required establishing the need for each element present in the object model against various elements extracted from the set of use case narratives. Although the system attempted to match variations in element labels, some of the domain specific synonyms were not recognized. Finally, due to certain inherent limitations of the underlying knowledge base (which was based on the collaboration patterns), the prototype system did not perform well with respect to handling alternative forms of modeling such as many-to-many relationships and class hierarchies.

We tested the changes in model consistency by conducting repeated MANOVA tests using SPSS 15.0. Table 2 shows the means and standard deviations of the required and invalid elements in the draft and final versions of the object models created by the subjects. The multivariate tests show a significant main effect for the participants' attempts ($F = .227$, $F(6,71) = 40.22$, $p < .001$). While, as expected, the final versions (NK2 and K2) contained relatively more required elements compared to the draft versions (with significant [$p < .001$] improvement in classes, attributes, and relationships identified), the average numbers of invalid or incorrect elements also increased significantly in both the treatment and the control groups. However, the interaction effect between treatment and attempt was non-significant ($F = 0.900$, $F(6,71) = .32$, $p = 0.260$). A further analysis showed a significant interaction only with respect to invalid classes and invalid relationships ($p = .030$ and $p = .044$, respectively), all other interactions were non-significant. In other words, whereas the OMA did not necessarily help in improving the consistency between the object models and the use cases in terms of required elements (as compared to participants not provided with the OMA), the OMA seemingly prevented participants from adding excessively large numbers of invalid classes and relationships.

Table 1
Assessment of the recommendations.

	Recommendations	Useful recommendations	Usefulness ratio (%)
Missing classes	2.66 (1.39)	1.68 (1.08)	65.9
Missing attributes	1.07 (1.01)	0.76 (0.86)	70.4
Missing relationships	19.76 (12.35)	0.39 (0.83)	3.4
Invalid classes	6.07 (1.82)	2.34 (1.61)	35.7
Invalid attributes	22.59 (7.05)	9.20 (4.91)	38.9
Invalid relationships	1.41 (2.03)	1.37 (2.03)	95.0

Table 2
Effect of OMA on Required and Invalid Elements of Object Models.

	Version	# of classes		# of attributes		# of relationships	
		Required	Invalid	Required	Invalid	Required	Invalid
Without OMA (<i>n</i> = 37)	NK1	8.86 (1.32)	4.57 (2.67)	17.76 (5.58)	21.97 (7.87)	4.92 (1.71)	9.19 (3.44)
	NK2	9.97 (0.87)	6.27 (2.85)	21.81 (4.97)	30.22 (9.55)	6.35 (1.51)	17.27 (7.00)
With OMA (<i>n</i> = 41)	K1	8.73 (1.03)	4.27 (2.24)	18.1 (4.93)	17.63 (7.13)	4.76 (1.65)	8.34 (4.65)
	K2	9.59 (1.09)	4.98 (1.74)	21.61 (5.00)	24.54 (7.09)	5.95 (1.90)	13.73 (5.02)

6. Discussion

This study aimed to address the problem of how to assist systems analysts in developing object models that are complete and valid with respect to a given set of requirements implied by the use case narratives. To this end, we developed a methodology and implemented it as a knowledge-based system prototype integrated within a UML modeling tool. We then conducted a controlled experiment to evaluate the utility and efficacy of the developed system.

The implementation of OMA follows an integrated methodology that makes use of NLP techniques to analyze a set of use case narratives and analysis patterns and then offer guidance by suggesting missing and invalid elements in a given object model. This prototype system supports the incremental and iterative development of object models, which is a common practice within use case driven systems development. To our knowledge, this innovative approach and its implementation is unique considering the available forms of support provided to analysts in object modeling.

The results of our analysis highlight three interesting factors related to the performance of our OMA prototype system. First, the system is relatively more effective in identifying invalid elements than missing elements. Although the use case narratives are expected to contain information related to the required elements, processing step descriptions using NLP techniques and identifying all the required elements needed to match these to the elements contained in the object model is a challenging process. On the other hand, determining whether or not an element present in a given object model is valid can be performed by simply going through the glossary of terms created by parsing use case narratives. Second, the prototype system performed poorly in identifying missing relationships, as it attempted to generate all possible relationships for all the pattern player types of a given class. Consequently, the number of recommendations for missing relationships far exceeded those of other types of recommendations. Clearly, the heuristics we employed for this purpose were not effective in pruning inapplicable recommendations related to missing relationships. Thus, it is important to incorporate additional heuristics for eliminating redundant and distant relationships. The third factor relating to performance is not directly related to the prototype system. According to the independent assessment of the usefulness of recommendations, a number of participants did not follow or incorporate many of the useful recommendations. It appears that the participants placed greater ef-

fort on removing invalid elements than on incorporating missing elements. A possible reason for this could be the relatively higher challenge in incorporating recommendations related to missing elements. More specifically, evaluating the usefulness of recommendations related to missing elements is cognitively more challenging than evaluating the usefulness of recommendations related to invalid elements. Furthermore, as the participants were under a time constraint, they were precluded from thoroughly analyzing each recommendation provided. Finally, the results show that, overall, the complexity of the models grew (in terms of required and invalid elements), which suggests that the participants expended considerable effort in attempting to enhance the completeness of the diagrams by adding more elements, rather than simply working through the system's recommendations.

As any study, this study is not without its limitations. A primary limitation may have been the fact the students worked on a very confined problem for a limited time. However, as we sought to develop a knowledge-based system targeted at typical systems analysts, our participant pool appears to have been adequate. Although the object modeling task was pilot tested to ensure that the duration of the experiment was adequate, this was not sufficient, especially for incorporating several of the useful recommendations offered by the system.

With the implementation of the present version of the OMA, we considered the completeness and validity aspects of consistency to ensure that the object models created truly represented the requirements implied by the use case narratives. As the test of the current prototype was geared towards demonstrating the efficacy of the proposed methodology and the system's ability to provide guidelines to help improve object models, we did not directly test the OMA against other alternative tools. However, we consider this to be a good direction for future research.

Notwithstanding these limitations, this study has several important implications for research and practice. First, the prototype system can be enhanced through further development. For example, by refining the knowledge base, making use of domain-specific ontologies, or including other aspects of model consistency. Similarly, to further reduce the cognitive load experienced by systems analysts, future research could identify ways to enhance the way recommendations are presented or to prompt the analyst to focus on the recommendations deemed most important (e.g., by presenting the recommendations one-by-one, and re-examining the model after any changes applied). Such extensions and additional evaluations would help to enhance the overall

quality of support provided to systems analysts engaging in object modeling. Furthermore, the scope of the prototype system can be expanded to other types of UML artifacts, such as activity diagrams and sequence diagrams, that have similar dependencies on use case narratives. The practical implications of this study are relevant to various groups of stakeholders involved in information systems development. Project teams could employ similar tools or techniques to establish standard notations and guidelines for the development of quality object models. This type of system could also be used for teaching object modeling and for training novice systems analysts in object modeling. Finally, CASE tool developers could use our proposed methodology to provide intelligent support for systems analysts.

7. Conclusion

Following a design science approach, we have created OMA as an extension to the ArgoUML open-source CASE tool. The OMA uses natural language processing to analyze use case narratives and employs heuristics to provide recommendations for improving the consistency of object models with respect to use case narratives. Results obtained from the evaluation show that the system can reliably extract information from use cases, thereby helping systems analysts to identify missing and invalid elements in object models. However, our analysis has also shown that the participants in our evaluation experiment did not incorporate some useful recommendations offered by the prototype, especially those contributing towards minimizing missing elements in the object models. Future research could attempt to improve the prototype system in this regard, or attempt to find ways to ensure that the analysts carefully consider the recommendations provided by the system.

The prototype presented in this study shows that automated tools can provide valuable guidelines for improving consistency of object models against a set of use case narratives, and thereby facilitate the development of better quality object models. Furthermore, the prototype's capacity to analyze use case narratives enables it to be used across different domains and give recommendations specific to the problem at hand. Hence, we see this extended system as being very valuable in business contexts. Another potential application would be to use this system in a learning context, especially for novice analysts. While we have demonstrated that the prototype system can be used to improve object models, future research should explore how the guidance provided affects learning, especially in comparison to generic guidelines or other approaches.

Acknowledgements

The work described in this paper was substantially supported by a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China (Project No. CityU 110207). The work of the third author has been partly supported by Sogang Business School's World Class University Program (R31-20002) funded by Korea Research Foundation.

Appendix A. SmallBytes subscription system case study

A small independent software journal, SmallBytes, has asked you to design a new system for managing its subscriptions and editorials. While the concept of managing subscriptions is quite straightforward, the details are numerous, as described below:

SmallBytes is published on a monthly basis; a typical monthly issue consists of 5–10 articles, each written by one or more authors in the software engineering field. Though the authors receive no payment for their articles, they do receive a year's free subscription

as a token of appreciation for their efforts. If they already have a subscription, then the expiration date is extended for a year.

SmallBytes has an editorial board of advisors, some of whom may also be authors from time to time. Each advisor on the editorial board normally serves for a one-year or two-year term, for which they receive a complimentary subscription to the magazine. Three or four members of the editorial board, selected by the editor, review each submitted article and provide their comments to the editor. As with most magazines, issues are scheduled and planned months in advance.

SmallBytes is managed by an editor supported by an assistant who is responsible for keeping track of subscriptions and submissions using an in-house computerized system. The assistant receives subscription orders (new and renewal) and new article submissions via mail or fax, and registers those orders and submissions into the system. The editor does a quick review of the

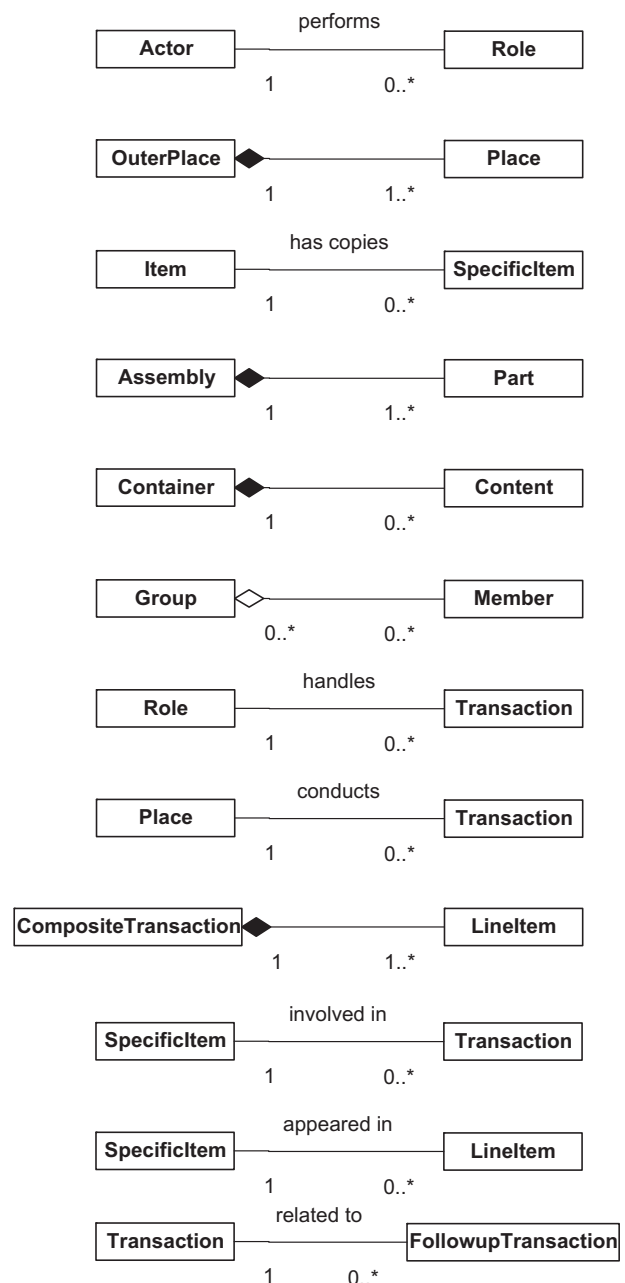


Fig. B1. Collaboration patterns.

Use Case	Process subscription request
Primary Actors	Assistant
Pre-conditions	Valid payment is attached to the subscription request
Post-condition	Subscriber's account exists in the system Subscription is saved Subscription number is generated Amount due is calculated Receipt is generated and sent to subscriber
Trigger	The publisher receives a subscription request
Main Success Scenario	1. Assistant enters the subscriber ID 2. Assistant enters the subscription period and the number of copies 3. System computes number of the monthly issues and subscription amount 4. Assistant enters the received amount 5. Assistant enter the check number, and check bank name 6. System logs subscription request, the payment received and the time of the transaction 7. System generates receipt
Extensions	1a. Subscriber ID does not exist System prompts for the subscriber's name, phone number, and address Assistant enters the subscriber details System generates an ID to the subscriber Return to Step 2 4a. Received amount does not match the subscription amount System informs the assistant about the difference in amount Return to Step 1 5a. Subscriber pays by credit card Assistant enters the amount due as received amount, credit card number, the expiry date, and the authorization code Return to Step 6

Fig. C1. Example of a use case narrative.

submissions to ensure that they fit with the magazine before assigning three or four advisors for detailed reviews and comments. Upon receiving replies from the advisors, the assistant records the review comments into the system. From time to time, upon receiving instructions from the editor, the assistant updates the list of advisors and their subject areas.

SmallBytes is sold on a subscription basis. Most subscriptions are for a one-year period, but the publisher accepts subscriptions for periods longer, or shorter than a year by simply pro-rating the annual subscription price. Payments for new subscriptions are normally received by cheque. Some subscribers pay by credit card, but the publisher insists (because its bank insists) that credit card payments be accompanied by a signature. This means that credit card orders are typically sent by fax or mail. As noted earlier, contributing authors and members of the editorial board of advisors receive a complimentary one-year subscription to the magazine.

Appendix B. Collaboration patterns

Collaboration patterns (Nicola, Mayfield, & Abney, 2002) involve pairs of classes belonging to the four types of pattern players: *people, places, things, and events*. The set of 12 primary patterns can be effectively employed to identify relationships (other than generalization-specialization) during the development of object models (see Fig. B1).

Appendix C. Heuristic operating on an example use case narrative

Assuming that the current version of an object model for the SmallBytes subscription system includes the classes Subscriber and Subscription, the first heuristic listed in Section 3 results in a recommendation for a missing relationship between Subscriber

and Subscription because these entities in the object model appear in the above use case narrative. If any one of these classes is not identified in the object model, then the prototype system will make a recommendation of missing classes. Similarly, other types of recommendations concerning missing and/or invalid attributes are provided by the system based on the information present in this narrative (see Fig. C1).

References

- Antony, S. R., & Batra, D. (2002). CODASYS: A consulting tool for novice database designers. *ACM Sigdis Database*, 33(3), 54–68.
- ArgoUML (2006). *ArgoUML version 0.20*.
- Batra, D. (2005). Conceptual data modeling patterns: Representation and validation. *Journal of Database Management*, 16(2), 84–106.
- Bolloju, N. (2004). Improving the quality of business object models using collaboration patterns. *Communications of the ACM*, 47(7), 81–86.
- Bolloju, N., & Sugumaran, V. (2012). A knowledge-based object modeling advisor for developing quality object models. *Expert Systems With Applications*, 39(3), 2893–2906.
- Borstler, J., & Hadar, I. (2008). Pedagogies and tools for the teaching and learning of object oriented concepts. *Lecture Notes in Computer Science*, 4906, 182.
- Campbell, L. A., Cheng, B. H., McUmber, W. E., & Stirewalt, R. E. (2002). Automatically detecting and visualising errors in UML diagrams. *Requirements Engineering*, 7(4), 264–287.
- de la Vara, J. L., & Sánchez, J. (2008). Improving requirements analysis through business process modelling: A participative approach. In *Proceedings of the 11th international conference on business information systems (BIS 2008). Lecture notes in business information processing* (pp. 165–176). Berlin, Heidelberg: Springer.
- Dobing, B., & Parsons, J. (2006). How UML is used. *Communications of the ACM*, 49(5), 109–113.
- Egyed, A. (2004). Consistent adaptation and evolution of class diagrams during refinement. In *Fundamental approaches to software engineering. Lecture notes in computer science* (Vol. 2984, pp. 37–53). Berlin, Heidelberg: Springer.
- Egyed, A. (2006). Instant consistency checking for the UML. In *Proceedings of the 28th international conference on Software engineering* (pp. 381–390). New York, NY: ACM.
- Eriksson, H. E., Penker, M., Lyons, B., & Fado, D. (2004). *UML 2 toolkit*. Indianapolis, IN: Wiley.
- Eshuis, R., & Wieringa, R. (2004). Tool support for verifying UML activity diagrams. *IEEE Transactions on Software Engineering*, 30(7), 437–447.

- Friedman-Hill, E. (2005). Jess version 7.0b6. <<http://www.eprints.agentlink.org/5380/>>.
- Gelbard, R., Te'eni, D., & Sade, M. (2010). Object-oriented analysis: Is it just theory? *IEEE Software*, 27(1), 64–71.
- Hakala, M., Hautamaki, J., Koskimies, K., Paakki, J., Viljamaa, A., & Viljamaa, J. (2002). Annotating reusable software architectures with specialization patterns. In *Proceedings of the working IEEE/IFIP conference on software architecture* (pp. 171–180).
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
- Kaindl, H. (2004). Active tool support for requirements engineering through RETH. In *Proceedings of the 12th IEEE international requirements engineering conference* (pp. 362–363). Los Alamitos, CA, USA: IEEE Computer Society.
- Kotb, Y., & Katayama, T. (2005). Consistency checking of UML model diagrams using the xml semantics approach. In *Special interest tracks and posters of the 14th international conference on World Wide Web* (pp. 982–983). New York, NY: ACM.
- Kuechler, B., & Vaishnavi, V. (2008). On theory development in design science research: Anatomy of a research project. *European Journal of Information Systems*, 17(5), 489–504. doi:10.1057/ejis.2008.40.
- Lange, C. F. (2006). Improving the quality of UML models in practice. In *Proceedings of the 28th international conference on Software engineering* (pp. 993–996). New York, NY: ACM.
- Larman, C. (2001). *Applying UML and patterns: An introduction to object-oriented analysis and design*. Prentice Hall PTR.
- Lucas, F. J., Molina, F., & Toval, A. (2009). A systematic review of UML model consistency management. *Information and Software technology*, 51(12), 1631–1645.
- March, S. T., & Smith, G. F. (1995). Design and natural science research on information technology. *Decision Support Systems*, 15(4), 251–266.
- Mens, T., Van der Straeten, R., & Simmonds, J. (2005). A framework for managing consistency of evolving UML models. In H. Yang (Ed.), *Software evolution with UML and XML* (pp. 1–30). Hershey, PA: Idea Group.
- Morisio, M., Travassos, G. H., & Stark, M. E. (2000). Extending UML to support domain analysis. In *Proceedings of the 15th IEEE international conference on Automated software engineering* (pp. 321–324). Los Alamitos, CA: IEEE Computer Society.
- Moritz, S. H., & Blank, G. D. (2005). A design-first curriculum for teaching Java in a CS1 course. *ACM SIGCSE Bulletin*, 37(2), 89–93.
- Nentwich, C., Capra, L., Emmerich, W., & Finkelstein, A. (2002). xlinkit: A consistency checking and smart link generation service. *ACM Transactions on Internet Technology (TOIT)*, 2(2), 151–185.
- Nicola, J., Mayfield, M., & Abney, M. (2002). *Streamlined object modeling: Patterns, rules, and implementation*. Upper Saddle River, NJ: Prentice Hall PTR.
- Overmyer, S. P., Lavoie, B., & Rambow, O. (2001). Conceptual modeling through linguistic analysis using LIDA. In *Proceedings of the 23rd international conference on software engineering, ICSE 2001* (pp. 0401). New York, NY: ACM.
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.
- Purao, S. (1998). APSARA: A tool to automate system design via intelligent pattern retrieval and synthesis. *ACM SIGMIS Database*, 29(4), 45–57.
- Purao, S., Storey, V. C., & Han, T. (2003). Improving analysis pattern reuse in conceptual design: Augmenting automated processes with supervised learning. *Information Systems Research*, 14(3), 269–290.
- Robbins, J. E., & Redmiles, D. F. (1998). Software architecture critics in the argo design environment. *Knowledge-Based Systems*, 11(1), 47–60.
- Robbins, J. E., & Redmiles, D. F. (2000). Cognitive support, UML adherence, and XML interchange in Argo/UML*. 1. *Information and Software Technology*, 42(2), 79–89.
- Schenk, K. D., Vitalari, N. P., & Davis, K. S. (1998). Differences between novice and expert systems analysts: What do we know and what do we do? *Journal of Management Information Systems*, 15(1), 9–50.
- Shanks, G., Tansley, E., & Weber, R. (2003). Using ontology to validate conceptual models. *Communications of the ACM*, 46(10), 85–89.
- Siau, K., & Cao, Q. (2001). Unified modeling language: A complexity analysis. *Journal of Database Management*, 12(1), 26–34.
- Siau, K., Erickson, J., & Lee, L. Y. (2005). Theoretical vs. practical complexity: The case of UML. *Journal of Database Management*, 16(3), 40–57.
- Sugumaran, V., & Storey, V. C. (2002). Ontologies for conceptual modeling: their creation, use, and management. *Data & Knowledge Engineering*, 42(3), 251–271.
- Sugumaran, V., & Storey, V. C. (2006). The role of domain ontologies in database design: An ontology management and conceptual modeling environment. *ACM Transactions on Database Systems (TODS)*, 31(3), 1064–1094.
- Viljamaa, A. (2001). *Pattern-based framework annotation and adaptation: A systematic approach*. Helsinki, Finland: University of Helsinki.
- Wand, Y., & Weber, R. (2002). Research commentary: Information systems and conceptual modeling – A research agenda. *Information Systems Research*, 13(4), 363–376.
- Wikipedia (2011, October 20). Comparison_of_Unified_Modeling_Language_tools. Wikipedia, The Free Encyclopedia. Wikimedia Foundation. <http://www.en.wikipedia.org/w/index.php?title=Special:Cite&page=List_of_Unified_Modeling_Language_tools&id=456479618>.
- Wohed, P. (2000). Tool support for reuse of analysis patterns—A case study. In *Conceptual modeling – ER 2000. Lecture notes in computer science* (Vol. 1920, pp. 591–660). Berlin, Heidelberg: Springer.